

SMART HEALTHCARE IoT SYSTEM

Integrated ESP32, FreeRTOS and Adafruit IO Project

Internship Project Report

Prepared by

Hariharasudan R

Electronics and Communication Engineering

SASTRA Deemed University

Project Platform: ESP32 + Wokwi + Adafruit IO

1. Abstract

This project implements a unified Smart Healthcare IoT system using an ESP32 microcontroller, FreeRTOS multitasking and Adafruit IO cloud connectivity. The system integrates environmental and health-related monitoring, medication dosage control, intelligent bed positioning, dynamic sampling, OLED visualization, and alert generation into one project. Sensor data such as air quality and oxygen level are acquired and processed by dedicated FreeRTOS tasks, displayed locally, and published to Adafruit IO. The system also includes network-state handling and an offline data-buffering approach to improve reliability during connectivity problems.

2. Introduction

Healthcare monitoring systems require continuous sensing, reliable communication, timely alerts and simple control interfaces. The objective of this project is to combine these requirements in a single embedded IoT platform. ESP32 provides the processing and wireless communication capability, while FreeRTOS is used to separate sensing, display, actuator, alert, MQTT and network responsibilities. Adafruit IO provides cloud feeds and dashboard controls for remote monitoring and control.

3. Objectives

- Integrate all internship tasks into one unified Wokwi project.
- Implement multitasking using FreeRTOS.
- Monitor AQI and oxygen level and publish live measurements.
- Provide medication dosage control with confirmation for high dosage values.
- Provide remote bed-position control through a servo.
- Implement dynamic sampling behavior for abnormal conditions.
- Provide OLED status, LED and buzzer alerts.
- Detect connectivity states and buffer data during offline operation.

4. System Overview

The integrated system consists of an ESP32 connected to simulated sensors and actuators in Wokwi. AQI and oxygen sensors provide input data. The OLED provides local feedback, while a servo represents the intelligent bed actuator. An LED and buzzer provide alert indications. Adafruit IO feeds provide cloud-side monitoring and control.

5. Hardware and Software

Category	Components / Tools
Controller	ESP32
Sensors	AQI sensor, Oxygen sensor
Display	128×64 OLED (SSD1306)
Actuator	Servo motor
Alerts	LED and buzzer
Cloud	Adafruit IO
Simulation	Wokwi
Programming	Arduino C/C++
RTOS	FreeRTOS

6. FreeRTOS Task Architecture

The software is divided into independent tasks so that sensing, user feedback, actuation, cloud communication and network management can operate as separate modules. A mutex is used for shared sensor/state data and a queue is used to transfer sensor information to the OLED task. A separate offline queue is used for temporary storage during connectivity loss.

Main Tasks

SensorTask: Reads AQI and oxygen inputs, converts the readings, updates shared variables and sends sensor data through the FreeRTOS queue.

OLEDTask: Displays AQI, oxygen, dosage, sampling information, bed information and system status.

ServoTask: Controls the bed-position servo according to the selected/required position.

AlertTask: Handles abnormal-condition indications using the LED and buzzer.

MQTTTask: Publishes monitored values such as heart rate, SpO2, body temperature, room temperature, oxygen and AQI to Adafruit IO.

NetworkTask: Maintains Adafruit IO connectivity, processes callbacks and handles network state.

7. Internship Task Implementation

Task 1 — Multi-Sensor Monitoring and FreeRTOS Integration

The project uses multiple FreeRTOS tasks to separate sensing, display, actuation, alerts, MQTT communication and network management. AQI and oxygen measurements are acquired by SensorTask and shared safely using a mutex and queue. This provides a modular structure for the integrated system.

Task 2 — IoT Health and Environmental Monitoring

The system publishes monitored parameters to Adafruit IO. The implemented feeds include heart rate, SpO2, body temperature, room temperature, oxygen level and AQI. The OLED provides local monitoring while the Adafruit IO dashboard provides remote visibility.

Task 3 — Smart Medication Dosage

A dosage slider on Adafruit IO is used as a remote control input. The callback receives the dashboard value and stores it as the target dosage. Values above the defined threshold require confirmation and the OLED shows a waiting-for-confirmation state. The LED and buzzer are used for alert indication when the defined critical dosage condition is reached.

Task 4 — Intelligent Remote Bed Control

The bed slider provides remote position control. The servo moves gradually to the requested position and returns or changes position according to the implemented control logic. Bed-position ranges are also used to display modes such as SLEEP, BREATHING, EMERGENCY and MANUAL on the OLED.

Task 5 — Dynamic Sampling Rate

The system supports manual sampling-rate selection and automatic fast sampling during abnormal conditions. The implemented emergency condition checks AQI, oxygen level and dosage. During an emergency condition the sampling interval is reduced to obtain more frequent monitoring. The manual sampling value is received from the Adafruit IO callback.

Task 6 — Network Fault Handling and Offline Buffering

The system maintains network state information and distinguishes normal operation from degraded or offline conditions. Sensor information can be placed into a FreeRTOS offline queue when connectivity is unavailable. A reconnection routine can process buffered information after connectivity is restored. Retry delays are increased between repeated connection attempts to reduce unnecessary connection traffic.

8. Adafruit IO Dashboard

The dashboard is used for both monitoring and control. The implemented feeds include AQI, oxygen level, heart rate, SpO2, body temperature, room temperature, dosage slider, bed slider, dosage confirmation and sampling-rate control. The dashboard therefore provides both live data visualization and remote control of the integrated system.

9. OLED User Interface

The OLED displays the current AQI and oxygen readings, servo/bed information, dosage, sampling interval, operating mode and alert/status messages. During high dosage confirmation, the OLED provides a clear WAITING FOR CONFIRMATION indication. Network-state messages are also used during offline operation.

10. Alert and Safety Logic

The alert mechanism responds to configured abnormal conditions. AQI above the defined threshold, low oxygen conditions and high dosage conditions can influence the alert and sampling behavior. The LED and buzzer provide immediate local indication while the OLED provides readable diagnostic information.

11. Data Flow

Sensor data flows from the simulated sensors to SensorTask, then through shared state and the sensor queue to the OLED and cloud publishing tasks. Dashboard controls such as dosage, bed position and sampling rate flow in the reverse direction through Adafruit IO callbacks to the ESP32 control logic.

12. Results

The integrated project was tested in Wokwi with Adafruit IO connectivity. The observed operation included live sensor updates, cloud feed updates, dosage callback handling, OLED status changes, servo movement, LED/buzzer alerts and dynamic sampling behavior. Network retry and offline-buffering mechanisms were also integrated into the project architecture.

13. Project Integration

All internship functions are implemented as modules within a single integrated ESP32 project. The same controller, communication layer, dashboard and user interface are shared across the functions. This avoids separate task demonstrations and represents the project as one complete Smart Healthcare IoT system.

14. Limitations

The Wokwi environment uses simulated sensor values rather than clinical-grade physical sensors. Accordingly, the system demonstrates embedded control, communication and software architecture rather than medical diagnosis. Dashboard feed limits and cloud rate limits also require efficient communication.

15. Future Enhancements

Future work could include real medical-grade sensors, persistent storage such as an SD card or flash, authentication improvements, encrypted communication, stronger state machines, event logging, historical analytics and integration with a dedicated healthcare application.

16. Conclusion

The Smart Healthcare IoT project demonstrates the integration of sensing, multitasking, cloud monitoring, remote control, alerts and fault handling in one ESP32-based system. FreeRTOS provides modular task execution while Adafruit IO provides cloud connectivity and dashboard interaction. The resulting system forms a unified platform suitable for demonstrating the six internship tasks as one integrated project.

17. Submission Checklist

Deliverable	Status
Single integrated Wokwi project	To attach final Wokwi link
GitHub repository	To attach repository link
Project Report PDF	Prepared
System Architecture Diagram	To add final diagram
Workflow / Block Diagram	To add final diagram
Dashboard Screenshots	To add screenshots
Demo Video	To record and upload

Note: Insert the final Wokwi link, GitHub link, dashboard screenshots, architecture diagram, workflow diagram and demo-video link before submitting the completed package.